

Keeping the Platform Running: A Maintenance Guide

For nextvillage / AI'ing & Vibing — written for someone who didn't come up through software
Written July 2026.

The Idea, in One Paragraph

You've spent your career around equipment that needs scheduled maintenance — filters get replaced, calibrations drift, parts wear out on a known timeline, and if you skip the schedule, something eventually fails, usually at a bad time. A software platform works the same way, just with different failure modes. Nothing here is a physical part wearing out, but the equivalent exists: AI models get discontinued by their makers, security keys age and become a liability, code libraries get patched for newly-discovered vulnerabilities, and services change their pricing or limits without much warning. None of this requires you to become a software engineer. It requires a maintenance schedule, the same way a piece of equipment does — and this document is that schedule.

The Maintenance Calendar

Task	Frequency	Why
Check for AI provider deprecation notices	Monthly	Models get retired on a schedule (see Section 1)
Skim npm dependency vulnerability warnings	Monthly	New security issues get discovered in libraries constantly (see Section 3)
Confirm error-alert email is still arriving	Monthly	The platform's early-warning system only helps if it's actually working (see Section 5)
Check AI provider spend against budget	Monthly	Costs can spike quietly, especially as user count grows (see Section 4)
Rotate the Supabase service-role key	Quarterly	It has full, unrestricted database access — the single most sensitive credential in the project (see Section 2)
Review who has repo/database access	Quarterly	People come and go; access should too (see Section 6)
Run <code>npm audit</code> and update outdated packages	Quarterly	Catches accumulated small issues before they compound (see Section 3)

Rotate remaining API keys (OpenAI, Groq, etc.)	Annual	Lower urgency than the Supabase key, but still good hygiene (see Section 2)
Confirm the domain registration hasn't lapsed	Annual	Domains expire on a fixed date; losing it is catastrophic and avoidable (see Section 7)
Manually click through 5-10 core user flows	Annual or after any large change	Automated checks verify the code builds, not that it feels right to use (see Section 5)

1. AI Models Change and Get Deprecated

This already happened once on this project — in the repo's history there's a commit titled "Migrate all Claude API calls from retired `claude-sonnet-4-20250514` to `claude-sonnet-4-6`." That's the pattern to expect: AI companies periodically retire specific model versions, usually with a few months' notice, and any code still pointing at the old model name starts failing outright once the retirement date passes.

This platform currently calls six different AI providers — Anthropic (Claude), OpenAI, Groq, Cerebras, Together, and Hugging Face — each with its own model names, each on its own deprecation schedule.

What to actually do

- Each provider publishes a "deprecations" page (search "[provider name] model deprecations"). Anthropic, OpenAI, and Groq all also email account owners in advance of a retirement if you're signed up with the account tied to the API key.
- Once a month, a quick check of these pages (or the inbox tied to each provider account) is enough — deprecations are announced well ahead of the actual cutoff, they're not surprises if you're checking.
- When a model does get retired, the fix is usually a small, mechanical change — updating a model-name string in one or two files, similar to the migration already done once. Not a rebuild.

If a page suddenly stops responding correctly and nothing else changed recently, "did a model we use just get retired?" is worth checking early — it's a common, easy-to-miss cause.

2. API Keys Are Credentials, and Credentials Age

Every API key is effectively a password that lets whoever holds it act as this project on that service — send AI requests billed to your account, send emails from your domain, or (in the case of the Supabase service-role key) read and write anything in the database, bypassing every permission check. The longer a key exists, the more places it's been copied into (local laptops, CI logs, old messages), and the more chances it's had to leak without anyone noticing.

Rotating a key means: generate a new one from the provider, update it in Vercel's environment variables, confirm the site still works, then delete the old one from the provider's dashboard so it stops working entirely.

The keys this project actually uses

Key	Priority	Why
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Highest	Full, unrestricted database access — bypasses all the row-level

		security rules that normally protect user data
<code>ANTHROPIC_API_KEY</code> , <code>OPENAI_API_KEY</code> , <code>GROQ_API_KEY</code> , <code>CEREBRAS_API_KEY</code> , <code>AI_TOGETHER_API_KEY</code> , <code>HF_TOKEN</code>	Medium	A leaked key means someone else can run up your AI usage bill
<code>RESEND_API_KEY</code>	Medium	A leaked key means someone could send email that appears to come from your domain
<code>CLOUDFLARE_API_TOKEN</code> , <code>SPEECHGEN_TOKEN</code> , <code>NEWS_API_SECRET</code>	Lower	Still worth rotating annually as routine hygiene

You don't need to remember this schedule yourself — this document is the reminder. Quarterly for the Supabase key, annually for the rest, is a reasonable cadence for a project this size.

3. Code Libraries Need Updates Too

This platform is built on roughly a thousand small, reusable pieces of code written by other people ("dependencies" or "packages") — everything from how the site talks to Supabase to how dates get formatted. Bugs and security holes get discovered in these regularly, and the maintainers ship fixes as updates. As of this writing, a routine install already flags **14 known vulnerabilities** (1 low, 6 moderate, 7 high severity) in the current dependency set — not an emergency, but a real, existing to-do item, not a hypothetical.

What to actually do

```
npm audit
```

Run from the project folder — it lists known issues and, for many of them, a fix that can be applied automatically. After running any fix, always re-run the test suite (`npm test`) and build (`npm run build`) before trusting the change, since dependency updates occasionally change behavior in ways that break something. This is exactly what the automated PR check now verifies for you.

4. Costs Can Grow Quietly

AI usage is billed per request, and as more students use the platform, that bill grows — usually gradually, but sometimes in a spike if, say, a feature gets used more heavily than expected or a bug causes repeated/unnecessary calls. This project already logs every AI call's estimated cost to a database table (`api_cost_log`), which means the data to catch a spike already exists — it just needs to be looked at periodically.

Each AI provider's own dashboard (Anthropic Console, OpenAI dashboard, etc.) also shows spend directly and can usually be configured to email you if usage crosses a threshold you set — worth setting

that up once, so you don't have to remember to check.

5. Knowing When Something's Actually Broken

The PR process and automated checks put in place recently verify that new code compiles and builds — they do not verify that a page still looks or behaves correctly once it's live, and they say nothing about a feature that was already live and quietly stopped working (the way the profile-completion popup did for six weeks before anyone noticed).

Some of this is already built and worth knowing about rather than rebuilding:

Already in place: when the platform logs an internal error to its `system_events` table, a webhook automatically emails an alert via Resend (see `api/triage-webhook.ts`). There's also a recurring daily activity report and a monthly assessment job already scheduled through Vercel. These exist — the maintenance task is simply confirming, periodically, that they're still actually firing (e.g. deliberately trigger a test error occasionally and confirm the email arrives), since a monitoring system nobody checks on is only slightly better than no monitoring system.

Beyond that, the most reliable check remains a human one: periodically — monthly is reasonable, always after a big change — actually sign in and click through the handful of flows that matter most (sign up, complete a profile, start a learning module, submit a community challenge). Five minutes catches things automated tools structurally can't.

6. Who Has the Keys to the Building

As the team grows beyond you, Silas, and Divinegift, periodically ask: does everyone who has access still need it? GitHub collaborator access and any Supabase dashboard access should be reviewed together — someone who's no longer active but still has push access or database access is a loose end worth tidying up, not an emergency, just routine.

7. The Boring, Easy-to-Forget Stuff

- **Domain registration.** `nextvillage.community` is registered somewhere and renews on a fixed annual (or multi-year) date. Losing track of this is one of the few genuinely catastrophic, entirely avoidable failure modes — the whole site becomes unreachable, and getting a domain back after it lapses and gets re-registered by someone else can be difficult or impossible. Whoever registered it should confirm auto-renewal is on and the payment method on file is current.
- **Backups.** Supabase takes automatic backups, but the retention window and how to actually restore from one depends on your plan tier — worth a one-time check of what's actually covered, so you're not learning the answer for the first time during an actual emergency.

When You're Stuck

None of this requires becoming a software engineer to execute — it requires recognizing when something on this schedule is due, and knowing that's a reasonable thing to bring to a future working session (with me, or with Silas/Divinegift as they get more comfortable in the codebase) rather than something you're expected to already know how to fix. The goal of this document is to make sure the right question gets asked at the right time, not to make you self-sufficient on every technical detail.

Companion to PR-Process-Guide.pdf. Both live outside the git repository for easy reference.